프로그래밍I - 관계기반

Chap #3 – 수학적 귀납법과 점화식

문제3 – Combination(S)

[문제 설명]

- ${}_nC_k$: n개 중 k개를 고르는 방법의 수
- ${}_nC_k = \frac{n(n-1)\ldots(n-k+1)}{k(k-1)\ldots 1} = \frac{n!}{k!(n-k)!}$
- n! 을 이용하는 경우 n이 커지면 overflow 발생(확인!)
- 정확하게 ${}_nC_k$ 를 구하는 프로그램 작성

  (단, 1 <= k <= n < 30)

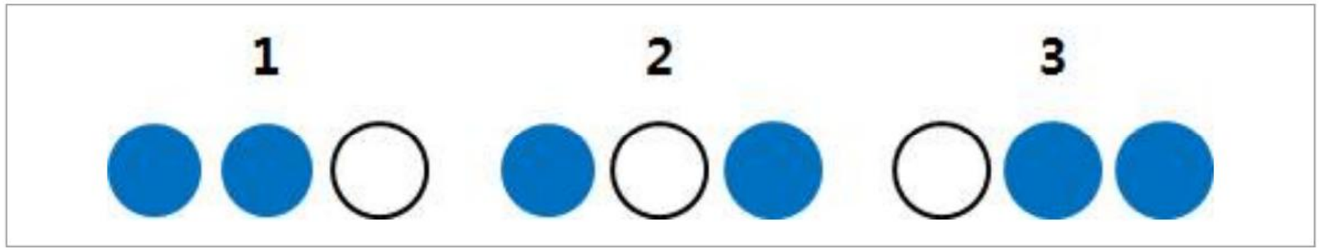
- 교재 p.29~34

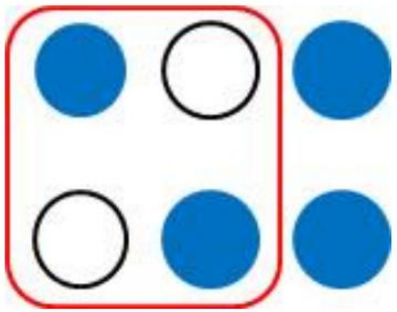
입력 값이 큰 경우(ex:300) 처리
=> p.87~ (동적표 활용)

Combination – 해결 전략 #1

[#1: 마지막 물건 선택 여부에 따라]

- 예> 3개 중 2개를 고르는 경우



마지막 물건을 고른 경우	마지막 물건을 고르지 않은 경우
	

Combination – 해결 전략 #1

[#1: 마지막 물건 선택 여부에 따라]

$$f(n, k) = \underbrace{f(n-1, k-1)}_{\text{마지막 물건을 고른 경우}} + \underbrace{f(n-1, k)}_{\text{마지막 물건을 고르지 않은 경우}}$$

1. f(n, k) 함수 정의: n개 중 k개를 고르는 경우의 수
2. 관계식

$$f(n, k) = \begin{cases} 1 & (n=k) \\ n & (k=1) \\ f(n-1, k-1) + f(n-1, k) & (1<k<n) \end{cases}$$

Combination – 해결 전략 #2

[#2: 수학 공식을 활용한 경우]

- f(n, k) = $\frac{n!}{k!(n-k)!}$ 이므로 f(n, k-1) = $\frac{n!}{(k-1)!(n-k+1)!}$
- f(n, k) = $\frac{n!}{k!(n-k)!}$ = $\frac{n!\ \ (n-k+1)}{(k-1)!\ k\ \ (n-k)!\ (n-k+1)}$

$$= \frac{n!}{(k-1)!(n-k+1)!} * \frac{(n-k+1)}{k} = \boxed{f(n, k-1) * \frac{(n-k+1)}{k}}$$

- 재귀 함수를 하나만 사용하여 결과를 얻을 수 있음
- 방법#1과 방법#2의 수행 시간 비교

Combination – 해결 전략 #3

[#3: 해결 전략 #2의 계산 순서 변경]

$_{70}C_{33}$

- 방법#2으로 계산 중 (1)에서 발생하는 overflow로 인해 계산 오류가 발생하는 경우를 줄이는 방법

$$f(n, k) = \underbrace{f(n, k-1) * (n-k+1)}_{(1)} / k$$

- 수식의 순서를 교체

$$f(n, k) = (n-k+1) / k * f(n, k-1)$$

- 주의! – 실수로 전환이 필요 (단, 실수 표현의 오차로 인해 값이 부정확)

ex) $f(10, 5) = f(10, 4) * (10-5+1) / 5$

$$f(10, 5) = \underbrace{(10-5+1) / 5}_{\text{(형 변환 필요)}} * f(10, 4)$$

$_{60}C_{30}$

Combination – 성능 비교

[codeup.kr]

2651	극장 좌석 배치 1
2652	극장 좌석 배치 2
2657	극장 좌석 배치 3

- 방법#1~3을 이용하여 n개 중 k개를 고르는 방법의 수를 구하는 프로그램을 작성하고, 결과 및 실행 시간을 비교하시오.

입력:	F(30, 10)		F(40, 10)		F(50, 10)	
	결과값	실행시간	결과값	실행시간	결과값	실행시간
방법#1	30045015		847660528		10272278170	
방법#2						
방법#3						

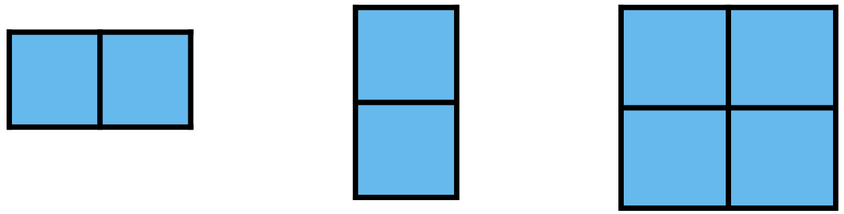
- 특징: 방법#1 - 설계가 간단(느림)
  방법#2 - 속도가 빠름(overflow),
  방법#3 - overflow 줄임(실수 표현 오차로 결과 부정확)

설계 간단 & 속도 빠름 &
결과 정확한 방법은??

문제4 – 타일 채우기

[문제 설명]

- 2*1 혹은 2*2 크기의 타일을 이용하여 2*n 크기의 직사각형 틀을 채우는 **경우의 수**를 구하시오.
  (단, 경우의 수가 커지므로 m으로 나눈 나머지 출력)



- 입력 범위: - 직사각형 틀의 가로 n (1 <= n <= 100,000)
  - 나누는 수 m (1 <= m <= 40,000)
- 교재 p.35~46

타일 채우기 – 해결 전략 #1

[#1: 끝부분을 채우는 방법에 따라]

$$f(n) = \underbrace{f(n-1)}_{\text{n-1 크기가 채워진 경우: 1가지}} + \underbrace{2*f(n-2)}_{\text{n-2 크기가 채워진 경우: 2가지}}$$

1. f(n) 함수 정의: 2*n 크기의 타일을 조건에 맞게 채운 경우의 수
2. 관계식

$$f(n) = \begin{cases} 1 & (n=0) \\ 1 & (n=1) \\ f(n-1) + 2*f(n-2) & (n>1) \end{cases}$$

OR

$$\begin{cases} 1 & (n=1) \\ 3 & (n=2) \\ f(n-1) + 2*f(n-2) & (n>2) \end{cases}$$

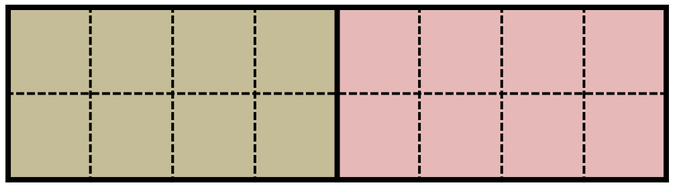
[간단히…]

$$\begin{cases} 1 & (n<=1) \\ f(n-1) + 2*f(n-2) & (n>1) \end{cases}$$

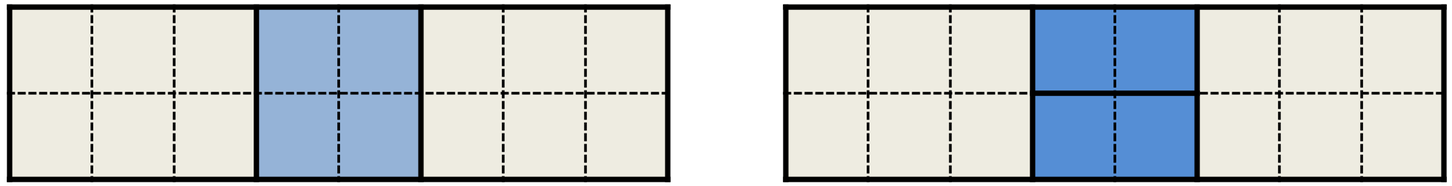
타일 채우기 – 해결 전략 #2

[#2: f(n/2)를 호출하는 개선된 관계식 – 짝수]

- $f(n) = \begin{cases} f(n/2)^2 + 2f(n/2-1)^2 & (n\text{:짝수}) \\ f(n-1) + 2f(n-2) & (n\text{:홀수}) \end{cases}$: 균등 분할 정복



$$f(\frac{n}{2}) * f(\frac{n}{2})$$



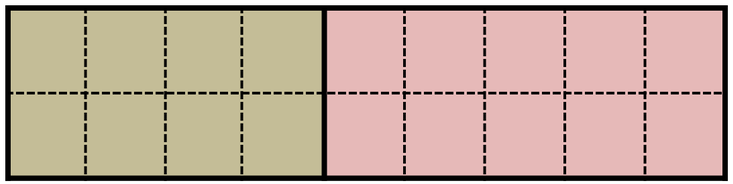
$$2 * f\left(\frac{n}{2} - 1\right) * f\left(\frac{n}{2} - 1\right)$$

타일 채우기 – 해결 전략 #3

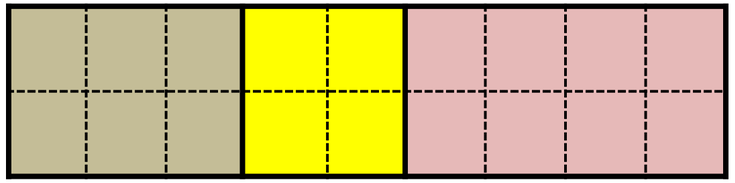
[#3: f(n/2)를 호출하는 개선된 관계식 - 홀수]

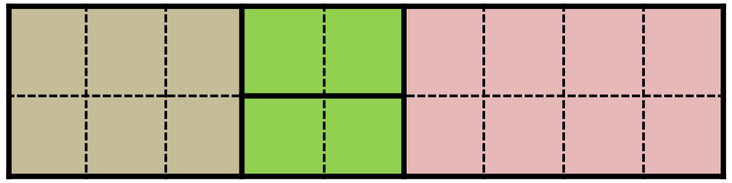
- f(n) = $\begin{cases} f(n/2)^2 + 2f(n/2-1)^2 & \text{(n:짝수) : 균등 분할 정복} \\ f([n/2])f([n/2]+1) + 2f([n/2]-1)f([n/2]) & \text{(n:홀수)} \end{cases}$

: 비균등 분할 정복



$$f([\frac{n}{2}]) * f([\frac{n}{2}]+1)$$





$$2 * f([\frac{n}{2}]-1) * f([\frac{n}{2}])$$

타일 채우기 – 성능 비교

- 방법#1~3을 이용하여 타일을 채우는 경우의 수를 구하는 프로그램을 작성하고, 결과 및 실행 시간을 비교하시오. (m=99999)

입력:	F(30)		F(100)		F(1000)	
	결과값	실행시간	결과값	실행시간	결과값	실행시간
방법#1	35041					
방법#2			15704			
방법#3					58139	

- 특징: 방법#1 – 설계가 간단(느림)
  방법#2 – 관계식 개선(짝수) : 빨라짐.
  방법#3 – 관계식 개선(짝수+홀수) : 훨씬 더 빨라짐.

메모이제이션을 적용하면??

연습문제를 풀어서 제출하세요!

[SASA-OJ]

- [대회]-[2026_2학기_프로그래밍I] 관계기반설계 연습문제 #3

- 기한 내에 제출하세요!
- 수업내용이나 과제 작성 과정에서 질문이 있다면 언제든 **질문**하세요!~
- E-mail:cray25@sasa.hs.kr

[codeup.kr]



3709	블럭 채우기 1
3712	블럭 채우기 2
3713	블럭 채우기 3
3714	블럭 채우기 4
3716	블럭 채우기 5
3719	블럭 채우기 6